

## LAB # 8

### Manipulation of String in java.

#### **Objective**

Introducing string class and to explore the strings class built-in methods.

#### **Lab Goals**

- Introduce the String class and its significance in Java programming.
- Understand essential methods of the String class such as length(), charAt(), substring(), and indexOf().
- Learn how to manipulate strings effectively using methods like concat(), toLowerCase(), and toUpperCase().
- Explore the usage of String comparison methods such as equals(), equalsIgnoreCase(), and compareTo().
- Introduction of StringBuffer and StringBuilder class in java.
- Master the concept of immutability in strings and its implications in programming.
- Practice solving problems involving string manipulation and processing using the String class and its methods.

#### **Theory**

##### **String Class In Java:**

The String class in Java represents a sequence of characters and offers various methods for text manipulation. The String class in Java is essential for handling text data. It provides methods for tasks like concatenation, substring extraction, and searching. Strings are immutable, ensuring thread safety and efficient memory management. Understanding the String class is crucial for effective text processing in Java.

##### **String Class built-in Methods**

| Method                | Description                                                       | Return Type |
|-----------------------|-------------------------------------------------------------------|-------------|
| charAt()              | Returns the character at the specified index (position)           | char        |
| compareTo()           | Compares two strings lexicographically                            | int         |
| compareToIgnoreCase() | Compares two strings lexicographically, ignoring case differences | int         |
| concat()              | Appends a string to the end of another string                     | String      |

|                    |                                                                                                                               |         |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|---------|
| contains()         | Checks whether a string contains a sequence of characters                                                                     | boolean |
| endsWith()         | Checks whether a string ends with the specified character(s)                                                                  | boolean |
| equals()           | Compares two strings. Returns true if the strings are equal, and false if not                                                 | boolean |
| equalsIgnoreCase() | Compares two strings, ignoring case considerations                                                                            | boolean |
| indexOf()          | Returns the position of the first found occurrence of specified characters in a string                                        | int     |
| isEmpty()          | Checks whether a string is empty or not                                                                                       | boolean |
| lastIndexOf()      | Returns the position of the last found occurrence of specified characters in a string                                         | int     |
| length()           | Returns the length of a specified string                                                                                      | int     |
| replace()          | Searches a string for a specified value, and returns a new string where the specified values are replaced                     | String  |
| substring()        | Extracts the characters from a string, beginning at a specified start position, and through the specified number of character | String  |
| toLowerCase()      | Converts a string to lower case letters                                                                                       | String  |
| toString()         | Returns the value of a String object                                                                                          | String  |
| toUpperCase()      | Converts a string to upper case letters                                                                                       | String  |
| trim()             | Removes whitespace from both ends of a string                                                                                 | String  |

**Example Code of charAt():**

```
public class CharAtExample {  
    public static void main(String[] args) {  
        String str = "Hello";  
        char result = str.charAt(1); // 'e'  
        System.out.println("Character at index 1: " + result);  
    }  
}
```

**Output:**

```
Character at index 1: e
```

**Example Code of compareTo():**

```
public class CompareToExample {  
    public static void main(String[] args) {  
        String str1 = "Apple";  
        String str2 = "Banana";  
        int result = str1.compareTo(str2); // Negative value  
        System.out.println("CompareTo result: " + result);  
    }  
}
```

**Output:**

```
CompareTo result: -1
```

**Example Code of compareToIgnoreCase():**

```
public class CompareToIgnoreCaseExample {  
    public static void main(String[] args) {  
        String str1 = "apple";  
        String str2 = "Apple";  
        int result = str1.compareToIgnoreCase(str2); // 0  
        System.out.println("CompareToIgnoreCase result: " + result);  
    }  
}
```

**Output:**

```
CompareToIgnoreCase result: 0
```

**Example Code of concat():**

```
public class ConcatExample {  
    public static void main(String[] args) {  
        String str1 = "Hello";  
        String str2 = " World";  
        String result = str1.concat(str2); // "Hello World"  
        System.out.println("Concatenated string: " + result);  
    }  
}
```

**Output:**

```
Concatenated string: Hello World
```

**Example Code of contains():**

```
public class ContainsExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        boolean result = str.contains("World"); // true  
        System.out.println("Contains 'World': " + result);  
    }  
}
```

**Output:**

```
Contains 'World': true
```

**Example Code of endsWith():**

```
public class EndsWithExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        boolean result = str.endsWith("World"); // true  
        System.out.println("Ends with 'World': " + result);  
    }  
}
```

**Output:**

```
Ends with 'World': true
```

**Example Code of equals():**

```
public class EqualsExample {  
    public static void main(String[] args) {  
        String str1 = "Hello";  
        String str2 = "Hello";  
        boolean result = str1.equals(str2); // true  
        System.out.println("Strings are equal: " + result);  
    }  
}
```

**Output:**

```
Strings are equal: true
```

**Example Code of equalsIgnoreCase():**

```
public class EqualsIgnoreCaseExample {  
    public static void main(String[] args) {  
        String str1 = "hello";  
        String str2 = "HELLO";  
        boolean result = str1.equalsIgnoreCase(str2); // true  
        System.out.println("Strings are equal ignoring case: " + result);  
    }  
}
```

**Output:**

```
Strings are equal ignoring case: true
```

**Example Code of indexOf():**

```
public class IndexOfExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        int result = str.indexOf("World"); // 6  
        System.out.println("Index of 'World': " + result);  
    }  
}
```

**Output:**

```
Index of 'World': 6
```

**Example Code of isEmpty():**

```
public class IsEmptyExample {  
    public static void main(String[] args) {  
        String str = "";  
        boolean result = str.isEmpty(); // true  
        System.out.println("String is empty: " + result);  
    }  
}
```

**Output:**

```
String is empty: true
```

**Example Code of lastIndexOf():**

```
public class LastIndexOfExample {  
    public static void main(String[] args) {  
        String str = "Hello World, Hello Java";  
        int result = str.lastIndexOf("Hello"); // 13  
        System.out.println("Last index of 'Hello': " + result);  
    }  
}
```

**Output:**

```
Last index of 'Hello': 13
```

**Example Code of length():**

```
public class LengthExample {  
    public static void main(String[] args) {  
        String str = "Hello";  
        int result = str.length(); // 5  
        System.out.println("Length of string: " + result);  
    }  
}
```

**Output:**

```
Length of string: 5
```

**Example Code of replace():**

```
public class ReplaceExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        String result = str.replace("World", "Java"); // "Hello Java"  
        System.out.println("Replaced string: " + result);  
    }  
}
```

**Output:**

Replaced string: Hello Java

**Example Code of substring():**

```
public class SubstringExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        String result = str.substring(6, 11); // "World"  
        System.out.println("Substring: " + result);  
    }  
}
```

**Output:**

Substring: World

**Example Code of toLowerCase():**

```
public class ToLowerCaseExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        String result = str.toLowerCase(); // "hello world"  
        System.out.println("Lowercase string: " + result);  
    }  
}
```

**Output:**

Lowercase string: hello world

**Example Code of toString():**

```
public class ToStringExample {  
    public static void main(String[] args) {  
        Integer num = 123;  
        String result = num.toString(); // "123"  
        System.out.println("Integer to string: " + result);  
    }  
}
```

**Output:**

```
Integer to string: 123
```

**Example Code of toUpperCase():**

```
public class ToUpperCaseExample {  
    public static void main(String[] args) {  
        String str = "Hello World";  
        String result = str.toUpperCase(); // "HELLO WORLD"  
        System.out.println("Uppercase string: " + result);  
    }  
}
```

**Output:**

```
Uppercase string: HELLO WORLD
```

**Example Code of trim():**

```
public class TrimExample {  
    public static void main(String[] args) {  
        String str = " Hello World ";  
        String result = str.trim(); // "Hello World"  
        System.out.println("Trimmed string: " + result);  
    }  
}
```

**Output:**

```
Trimmed string: Hello World
```

## Introduction to StringBuffer:

**StringBuffer** is a class in Java that is used to create mutable (modifiable) strings. Unlike String, which is immutable, StringBuffer allows for string modifications without creating new objects. It is synchronized, making it thread-safe, but this can make it slower than **StringBuilder**, which is its non-synchronized counterpart.

### Syntax:

To create a StringBuffer object, you can use the following constructors:

```
StringBuffer()  
StringBuffer(int capacity)  
StringBuffer(String str)
```

### Example Code:

```
public class StringBufferExample {  
    public static void main(String[] args) {  
        // Create a StringBuffer object  
        StringBuffer sb = new StringBuffer("Hello");  
  
        // Append text to the buffer  
        sb.append(" World");  
        System.out.println("After append: " + sb.toString());  
  
        // Insert text at a specified index  
        sb.insert(5, " Java");  
        System.out.println("After insert: " + sb.toString());  
  
        // Replace a portion of the string  
        sb.replace(5, 10, "Beautiful");  
        System.out.println("After replace: " + sb.toString());  
  
        // Delete a portion of the string  
        sb.delete(5, 14);  
        System.out.println("After delete: " + sb.toString());  
  
        // Reverse the string  
        sb.reverse();  
        System.out.println("After reverse: " + sb.toString());  
    }  
}
```

**Output:**

```
After append: Hello World
After insert: Hello Java World
After replace: Hello Beautiful World
After delete: Hello World
After reverse: dlrow olleH
```

**Introduction to StringBuilder:**

**StringBuilder** in Java is a class that provides a mutable sequence of characters. Unlike String, which is immutable, StringBuilder allows you to modify the sequence of characters in the string without creating a new object each time.

**Syntax:**

```
StringBuilder sb = new StringBuilder(); // Create an empty StringBuilder
StringBuilder sb = new StringBuilder("initial"); // Create a StringBuilder with initial value
```

**Example Code:**

```
public class StringBuilderExample {
    public static void main(String[] args) {
        // Create a StringBuilder object
        StringBuilder sb = new StringBuilder("Hello");
        // Append text to StringBuilder
        sb.append(" World");
        // Insert text at a specific position
        sb.insert(5, ", Java");
        // Delete characters
        sb.delete(5, 11);
        // Reverse the StringBuilder
        sb.reverse();
        // Convert StringBuilder to String
        String result = sb.toString();
        // Output the result
        System.out.println("Modified StringBuilder: " + result);
    }
}
```

**Output:**

```
Modified StringBuilder: dlrow ,olleH
```

**Exercise**

- A. Construct a program which shows the implementation of concat() string class built-in method.
- B. Construct a program which shows the implementation of compareTo() string class built-in method.
- C. Construct a program which shows the implementation of equals() string class built-in method.
- D. Construct a program which shows the implementation of length() string class built-in method.
- E. Construct a program which shows the implementation of toUpperCase() string class built-in method.
- F. Write a Java program that demonstrates the usage of StringBuffer. Your program should prompt the user to enter two strings, concatenate them using StringBuffer, and then reverse the concatenated string. Finally, output the reversed string.
- G. Write a Java program that demonstrates the usage of StringBuilder. Your program should initialize a StringBuilder object with the value "Java is", then insert the text "awesome" at index 7, and finally replace "awesome" with "powerful". Output the final string.